



A G E N T H A R N E S S

The core architecture for building next-generation AI agents



Engineering practice and design philosophy from Azure Foundry and Anthropic to AWS Bedrock

Kinfey Lo – Microsoft Senior Cloud Advocate

2026

Today's agenda

Six major topics covering the design and practice of Agent Harness in depth

- | | | |
|----|--------------------------------|---|
| 01 | Conceptual foundation | What is Agent Harness and its core components |
| 02 | The three platforms | Deep dive on Azure Foundry, Anthropic, and AWS Bedrock |
| 03 | Architecture comparison | Multi-dimensional comparison of isolation models, protocols, identity, and security |
| 04 | Practical guide | Dev workflow, context engineering, and performance optimization |
| 05 | Future outlook | The future of Meta-Harness and evolvable architectures |
| 06 | Q&A | Deep discussion and live Q&A |

PART 01

Conceptual foundation

Understanding Agent Harness: the key leap from Prompt to engineering



Why we need Agent Harness

Building production-grade AI agents involves many cross-cutting concerns — far beyond Prompt engineering

Containerized deployment

Image builds, web server config, runtime orchestration

Security governance

Identity management, least privilege, sensitive-data protection

State persistence

Session memory, file storage, cross-turn state recovery

Elastic scaling

Concurrent sessions, idle reclamation, cold-start latency

Observability

Log aggregation, metric collection, tracing and diagnostics

Version rollback

Canary releases, blue-green deployments, traffic shifting

Agent Harness is the engineering abstraction that converges all these complexities into the platform layer

What is Agent Harness

Harness = the execution environment, control loop, and infrastructure that surround the LLM

The three-layer abstraction of Agent Harness

Brain

LLM reasoning + Harness control loop

Hands

Sandbox execution environment + tool-call interface

Session

Append-only event log + state-recovery mechanism

Core definition

Agent Harness

An engineering framework built around the LLM — responsible for invoking the model, routing tool calls, maintaining state, and ensuring security.

OS analogy

Harness is to LLMs what an OS is to CPUs — through virtualization and abstraction, hardware complexity is converged into the platform layer.

Evolution pattern

A Harness encodes assumptions about LLM capabilities. As models grow stronger, some assumptions go stale — so interfaces should be stable while implementations can evolve.

Source: Anthropic Engineering — Scaling Managed Agents (2026)

The three core components of Agent Harness

Anthropic's virtualization model: decoupled Brain / Hands / Session design

Brain

LLM reasoning and control loop

- Calls Claude / GPT / in-house models for inference
- Parses output, routes tool calls
- Stateless design — horizontally scalable
- Restart via `wake(sessionId)` on failure

Hands

Sandbox and tool execution

- `execute(name, input) → string` — universal interface
- Containers, functions, MCP Servers, external APIs all work
- Pluggable — swap in Phone, emulator, or external system
- Failures are treated by Brain as ordinary tool-call errors

Session

Persistent event log

- Append-only event stream — records every interaction
- `getEvents()` lets Brain replay context
- Lives independently of Brain and container
- Supports crash recovery and cross-device access

The power of decoupling: separating Brain, Hands, and Session

Loosely coupled interfaces let any component fail, be replaced, and scale independently

× Coupled architecture (early approach)

- × Brain, Hands, and Session live in one container
- × Container fails → entire session lost
- × Cannot connect to customer's VPC resources
- × Hard to debug — failure causes hard to pinpoint
- × TTFT severely slowed by container cold starts
- × Credential leaks can be exploited by Prompt Injection

✓ Decoupled architecture (Managed Agents)

- ✓ Three components deploy independently — each can fail and restart on its own
- ✓ Brain is stateless — horizontal scaling is trivial
- ✓ Hands are created on demand only — they don't block TTFT
- ✓ Can connect to customer VPC — Brain needs no peering
- ✓ Credentials never enter the sandbox — accessed via proxy
- ✓ P50 TTFT down 60%, P95 down 90%+

"Don't adopt a pet" — treat containers like cattle, not pets

The three-stage evolution of AI Agents

From Prompt templates to managed runtimes — engineering capability grows step by step



Stage 1

Prompt template

Single-turn tasks via System Prompt + Few-shot



Stage 2

Custom Agent frameworks

LangChain / Semantic Kernel / Agent Framework — self-managed orchestration loop



Stage 3

Managed Agent platforms

Azure Foundry / Anthropic / AWS Bedrock offer Harness-as-a-Service

Current focus: Stage 3 — major cloud vendors compete fiercely on Managed Harness standards

PART 02

The three platforms in depth

Microsoft Foundry · Anthropic Managed Agents · AWS Bedrock



Overview of the three managed Agent platforms

Representing three different design philosophies and engineering trade-offs

**AZ**

Microsoft Foundry
Hosted Agents (Preview)

- BYOC: container-as-code, any framework
- Responses + Invocations dual protocols
- Per-session VM-isolated sandbox
- Microsoft Entra ID auto-bound
- Deep integration with Foundry Toolbox

**AN**

Anthropic Managed
Managed Agents

- Brain / Hands / Session three-layer decoupling
- Meta-Harness philosophy, stable interfaces
- Built for long-running tasks — sessions persist independently
- MCP tools and proxy-isolated credentials
- Detailed design published in the engineering blog

**AWS**

AWS Bedrock
Managed Agents (OpenAI)

- First commercial OpenAI Harness rollout
- Built on AgentCore default compute environment
- Data stays inside AWS — identity and audit built in
- Close to existing AWS data and compute services
- Limited Preview stage

Microsoft Foundry Hosted Agents

Microsoft's managed agent platform — supports bring-your-own framework and code

Platform positioning

Agents run as containerized applications on Microsoft-managed infrastructure. Developers retain full control over code and framework; the platform handles scaling, state, observability, and lifecycle.

Use cases

- ▶ Bring-your-own code / framework (Agent Framework, LangGraph, Semantic Kernel)
- ▶ Need custom protocol to receive Webhooks or non-OpenAI payloads
- ▶ Need control over compute resources (CPU/Memory)
- ▶ Need stateful workloads (cross-turn persistence)

BYOC

Bring-your-own container image

Auto-Scale

Elastic scaling + zero idle cost

VM Isolation

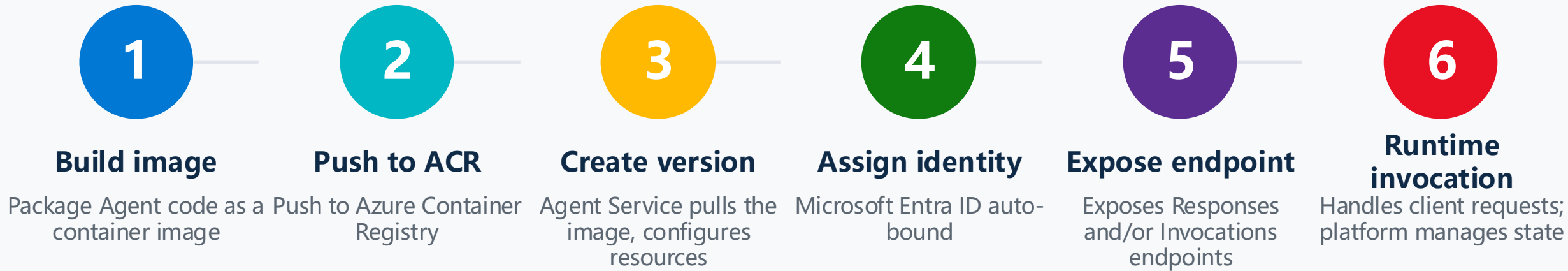
Per-session isolated VM sandbox

Entra ID

Dedicated identity and endpoint per Agent

How Azure Foundry works

The full lifecycle from image build to runtime invocation



Platform-managed automatically:

- Container lifecycle: provisioned on demand when active; auto-unloaded on idle timeout (15 minutes)
- State persistence: \$HOME and /files contents auto-restored on session recovery
- Elastic scaling: supports 0.25 vCPU / 0.5 GiB through 2 vCPU / 4 GiB sandboxes
- Observability: OpenTelemetry integration; logs, traces, and metrics built-in

Responses and Invocations: dual protocols

Two protocols cover conversation, webhook, batch, and custom scenarios

Responses protocol

Suited for	Conversational bots, multi-turn Q&A, RAG, background async
Format	OpenAI-compatible /responses protocol
Client	Any OpenAI SDK (Python / JS / C#) works out of the box
Session	Platform-managed conversation history (conversation ID)
Streaming	ResponseEventStream — built-in lifecycle events
Background	background: true + platform polling/cancel

Invocations protocol

Suited for	Webhook intake, classification/extraction, custom streaming protocols
Format	Arbitrary JSON in, arbitrary JSON out
Client	Custom SDK — you define the contract
Session	App-layer managed (memory / Cosmos DB)
Streaming	Native SSE — fully under your control
Background	Track tasks yourself and implement polling endpoints

Both protocols can run simultaneously, and can layer Activity (Teams/M365) and A2A (Agent-to-Agent) protocols on top

Agent identity and endpoints

Every managed Agent automatically gets a dedicated Entra ID and endpoint

Roles of the two identities

Microsoft Entra ID(Agent identity)

- Auto-created at deployment — independent per Agent
- Container-runtime auth — calls models and downstream services
- Auto-granted the Azure AI User role via azd

Project Managed Identity(project-level)

- System-assigned identity at the Foundry project
- Dedicated to platform-infrastructure operations
- e.g., Container Registry Reader
- Separate from the Agent's runtime identity

Auto-exposed endpoints

Responses Endpoint

```
{project_endpoint}/agents/{name}/  
endpoint/protocols/openai/v1/responses
```

Invocations Endpoint

```
{project_endpoint}/agents/{name}/  
endpoint/protocols/invocations
```

OBO (On-Behalf-Of): can call downstream services on behalf of the user when integrating Teams/M365

Sessions and Conversations

Two state abstractions serving different protocols

Dimension	Session	Conversation
Definition	Logical session containing \$HOME and /files	Conversation history stored in Foundry
Persistence	\$HOME and /files persisted across turns	Messages, tool calls, and responses
Lifecycle	30-day cap, unloaded after 15 minutes idle	Long-term storage independent of compute state
Dominant protocol	Invocations protocol-led	Responses protocol-led
Cross-channel	Bound to session ID	Shared across Playground / API / Teams
Management API	List / terminate / upload files	Auto-maintained by Foundry

Session compute lifecycle: Active → 15 min idle → state persisted → compute restored on next request with same ID

Security and data governance

Per-session VM isolation + multi-layer security boundaries

Per-Session VM Isolation

- ✓ Per-session isolated VM sandbox
- ✓ Persistent \$HOME and /files filesystems
- ✓ Scale-to-zero with stateful recovery
- ✓ Fully isolated between sessions

Credential and secret management

- ⚠ Never bake secrets into images or env vars
- ⚠ Use Managed Identity and Connections
- ⚠ Secrets centralized in Azure Key Vault
- ⚠ Private-network deployment requires manual RBAC for the Agent ID

Private network support

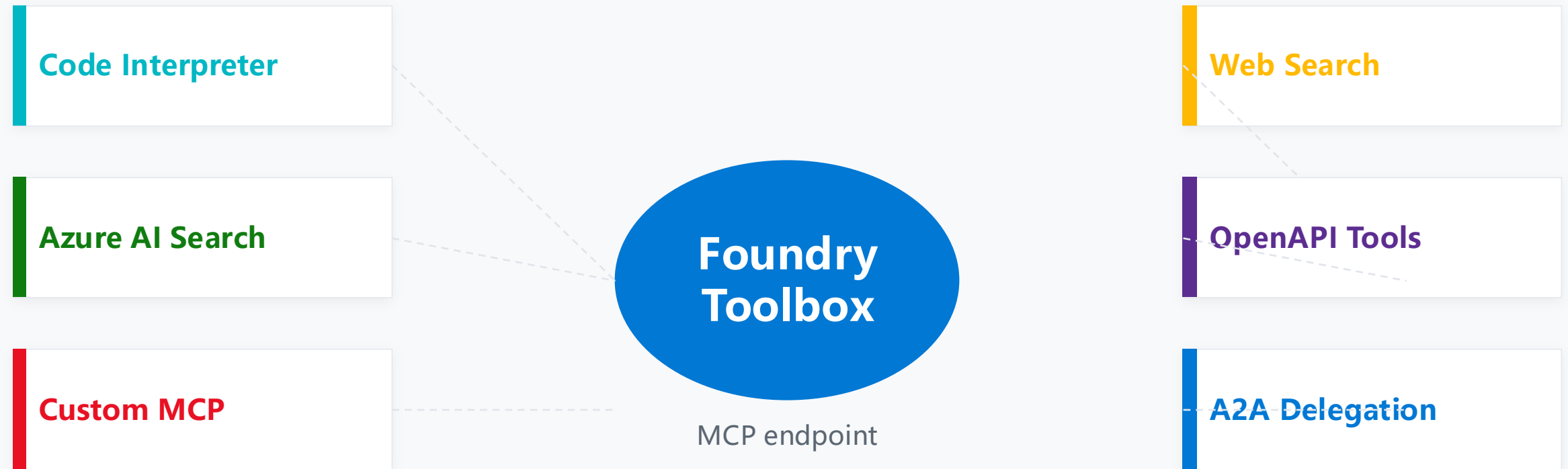
Hosted Agents can be deployed within network-isolated Foundry resources. Note: the Azure Container Registry hosting the image still requires a public endpoint — private-network ACR is not yet supported.

Third-party system risk

When connecting to non-Azure Direct Models or third-party MCPs, data may cross the Azure compliance boundary. Review data-sharing, retention, and location policies one by one.

Foundry Toolbox and platform extensions

Connect a rich built-in toolset via the MCP endpoint



Agent code connects to the Toolbox endpoint via a standard MCP client — the platform never auto-injects tools; developers retain full control

Anthropic Managed Agents

Claude Platform's managed service — built for long-running Agent tasks

"Harnesses encode assumptions that go stale as models improve."

A Harness encodes assumptions about model capability — assumptions that grow stale as models improve. Managed Agents are built around a stable interface set, so interfaces outlive any specific implementation.

01

Stable interfaces

Stable Interfaces

Like the OS read() — the top-level interface is fixed while the underlying implementation can evolve freely

02

Decoupled components

Decoupled Components

Three-way separation of Brain / Hands / Session — independent failure and replacement

03

Meta-Harness

Meta-Harness Design

Opinionated about interfaces — neutral about underlying implementations

Key APIs of the three interfaces

A minimal interface set lets components evolve independently

Brain	<code>wake(sessionId)</code> Wakes a new Harness process — recovers from the session	<code>execute(name, input) → string</code> Invokes any Hand — unified abstraction for tool calls	<code>emitEvent(id, event)</code> Writes persistent events to the session log
Hands	<code>execute(name, input) → string</code> Brain doesn't see implementation details	<code>provision({resources})</code> Provisions container/compute resources on demand	<code>Failure → tool-call error</code> Container death is handled by Brain via retry or routing
Session	<code>emitEvent(id, event)</code> Append-only event append	<code>getEvents()</code> Slice context by position to replay	<code>getSession(id)</code> Rebuilds Brain's execution context after a crash

Source: Anthropic Engineering Blog, "Scaling Managed Agents" (2026-04-08)

Structured security-boundary design

Credentials never enter the sandbox: Resource-Bundled Auth + Vault

× Risk: credentials and code share the sandbox

Prompt Injection only needs to convince Claude to read its own environment to obtain credentials. Once an attacker holds the Token, they can spin up unrestricted sessions and delegate work to them.

✓ Fix: physically isolate credentials

Structural fixes guarantee Tokens are never reachable from inside the sandbox. Even as Claude grows smarter, it cannot access any credentials from inside the sandbox.

Resource-Bundled Auth

Credentials bundled with resources

Git example: during sandbox init, use the repo Token to clone and inject the Token into the local git remote. push/pull works normally inside the sandbox — the Agent never handles the Token directly.

Vault + MCP Proxy

Credentials stored in a vault outside the sandbox

OAuth Tokens live in a secure Vault. Claude calls MCP tools through a dedicated proxy, which uses the session Token to fetch the matching credential and then calls downstream services. The Harness never sees the credential.

Many Brains, Many Hands architecture

Scaling dividends from decoupling: lower latency + VPC freedom

Many Brains

Once Brain is decoupled from the container, lots of Brains are just stateless processes — starting many Brains is just spawning many processes. Containers are provisioned only on demand via `execute()` tool calls — no more paying for container startup on sessions that never touch the sandbox.

Many Hands

The `execute(name, input) → string` interface can carry any tool: containers, MCP Servers, in-house tools. A Hand is not bound to a specific Brain — so Brains can pass Hands to each other.

Performance gains: substantial Time-To-First-Token improvement

60%

P50 TTFT down

90%+

P95 TTFT down

0

No needless container-startup waits

Session ≠ Claude's context window

Recoverable context storage + customizable context engineering

Bottlenecks of the traditional approach

- ✗ Long tasks exceed Claude's context window
- ✗ Compaction makes irreversible decisions
- ✗ Context trimming discards Tokens that may be needed later
- ✗ If not stored, compacted messages can't be recovered
- ✗ REPL keeps context but is bound to the sandbox's lifetime

Managed Agents' solution

- ✓ Session = persistent event stream — independent lifespan
- ✓ `getEvents()` exposes a position-slice interface
- ✓ Brain can replay any historical slice
- ✓ Harness is free to transform data before writing it into context
- ✓ Storage and context-engineering concerns are fully separated
- ✓ Also optimizes Prompt Cache hit rate

AWS Bedrock Managed Agents

AWS managed agent platform powered by OpenAI Harness

Bedrock Managed Agents, powered by OpenAI

Combines frontier AI models with trusted AWS infrastructure so customers can quickly build production-grade, OpenAI-powered agents in the cloud.

Frontier intelligence

Uses OpenAI Harness to unlock frontier-model potential — faster execution, sharper reasoning, controllable long tasks.

Secure by default

Every Agent has its own identity and every action is audited. Runs in the customer's environment — data never leaves AWS.

AWS scale

Connects directly to AWS compute and data services — close to enterprise-grade data and system deployments.

AgentCore integration

Bedrock AgentCore is the default compute environment — provides policy, discovery, and observability capabilities.

AWS Bedrock key capabilities in detail

Powered by the OpenAI Harness + AgentCore dual engines

OpenAI Harness core

Inference speed

Execution path deeply optimized for frontier models

Inference quality

Unlocks sharper model reasoning

Long-running tasks

Reliable long-duration control and state persistence

Runtime

Inference + memory + skills managed uniformly inside the customer environment

AgentCore default runtime environment

Open platform

Supports any model and framework for building Agents

Authorization policy

Policy enforcement + unified tool-permission management

Agent discovery

Service discovery for Agents and Agent Tools

Observability

Evaluation and ops monitoring scale with the enterprise

PART 03

Architecture comparison

Comprehensive comparison across isolation models, protocols, security, and observability



Three-platform architecture comparison matrix

Dimension × platform — one table for the key differences

Dimension	Azure Foundry	Anthropic	AWS Bedrock
Runtime mode	BYOC container	Managed decoupled components	Managed + AgentCore
Framework support	Agent FW / LangGraph / SK	Claude SDK	OpenAI Harness
Isolation unit	Per-session VM sandbox	Sandbox container (swappable)	Customer AWS environment
Primary protocol	Responses + Invocations	Brain/Hand/Session API	OpenAI-compatible
Identity model	Microsoft Entra ID	MCP Token + Vault	Per-Agent independent identity
State layer	Session + Conversation	Session event stream	Runtime-managed
Observability	OpenTelemetry built-in	Replayable event stream	AgentCore monitoring
Stage	Preview	Public release	Limited Preview

Containerization and isolation models compared

Three isolation philosophies map to three engineering trade-offs

Azure: VM-level session isolation

Per-session VM Sandbox

- Each session gets its own VM — extremely strong isolation boundary
- \$HOME and /files auto-persisted
- Scale-to-zero + state recovery together
- Predictable cold starts — suitable for enterprise compliance

Anthropic: containers as cattle

Container as Cattle

- Sandbox is just an execute() tool implementation
- Failures are handled by Brain as tool errors
- Hand can run inside the customer's VPC
- Brain decoupled from container — easy to swap

AWS: AgentCore default environment

Customer Environment

- Agent runs in the customer's AWS environment
- Data never leaves AWS
- Leverages AWS compute / storage / network
- AgentCore provides horizontal-scaling capability

Protocol and interface design philosophies

OpenAI-compatible vs. Meta-Harness internal API vs. dual-protocol hybrid

Azure: Dual protocols coexist

Advantages

- ✓ Responses works out of the box
- ✓ Invocations is flexibly customizable
- ✓ Layerable with Activity / A2A

Limitations

- ✗ Must pick a primary protocol
- ✗ Coexistence adds operational complexity

Anthropic: Internal-API design

Advantages

- ✓ Stable Brain/Hand/Session interfaces
- ✓ Designed for "programs yet to come"
- ✓ High evolvability

Limitations

- ✗ Tied to the Claude platform
- ✗ Requires familiarity with Managed Agents API

AWS: OpenAI-compatible first

Advantages

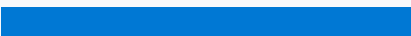
- ✓ Familiar ecosystem
- ✓ Clear migration path
- ✓ OpenAI Harness performance optimization

Limitations

- ✗ Centered on OpenAI models
- ✗ Cross-vendor interface not aligned

Security and identity management compared

Three approaches to credentials, isolation, and audit

Dimension	Azure	Anthropic	AWS
Per-Agent independent identity		 4/5	
Credential-isolation strength	 4/5		 4/5
Network-isolation capability	 4/5	 4/5	
OBO / delegated authorization		 3/5	 4/5
Audit-log completeness	 4/5	 4/5	

Note: scores are only relative indicators based on public documentation and design philosophy — not absolute quantification.

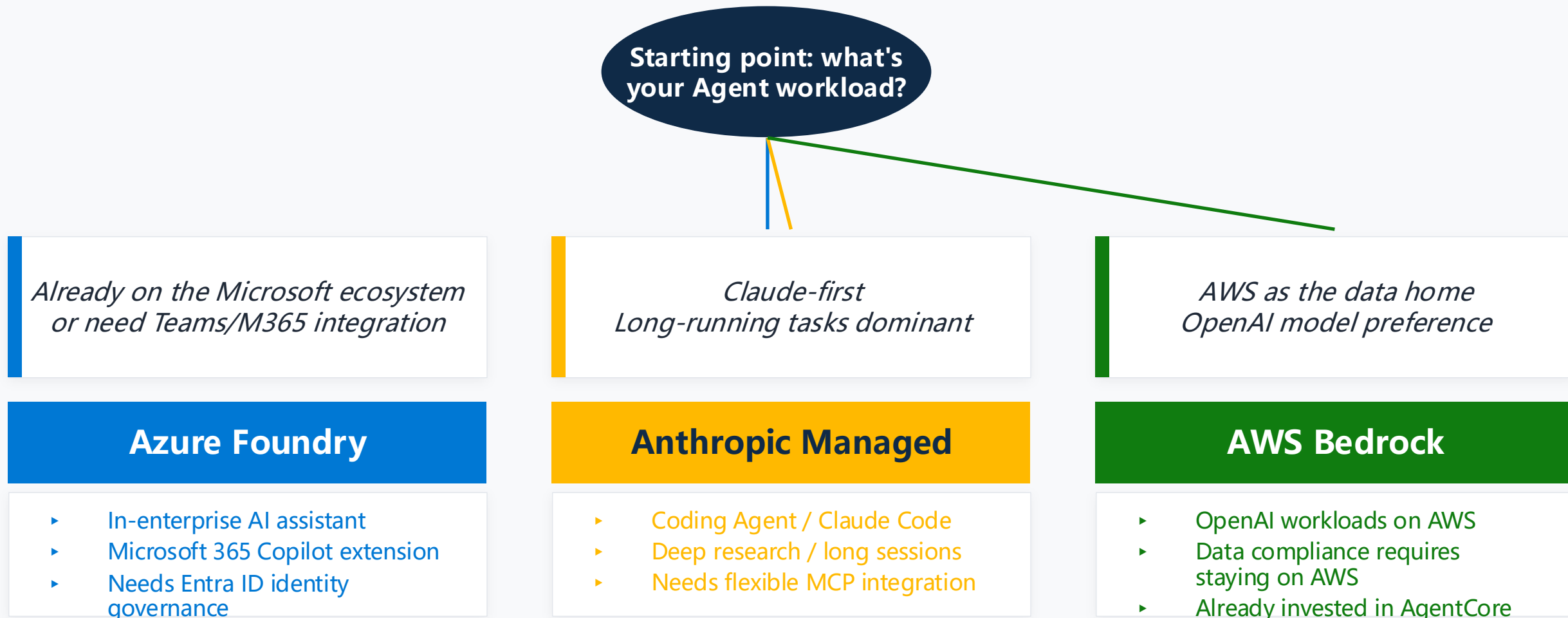
Observability, billing, and scalability compared

Three key operational differences

Observability	Azure OpenTelemetry + Foundry Tracing + Evaluators	Anthropic Session event log fully replayable	AWS AgentCore evaluation, monitoring, Agent discovery
Billing model	Azure Billed on active-session CPU + Memory consumption	Anthropic By Token + Sandbox resources	AWS By Bedrock inference + AgentCore resources
Scalability	Azure Default 50 concurrent sessions/subscription — quota expandable	Anthropic Brain is stateless — horizontal scaling is easy	AWS Aligned with AWS compute and data scale

Platform-selection decision framework

Three-step decision based on scenario, ecosystem, and compliance constraints



PART 04

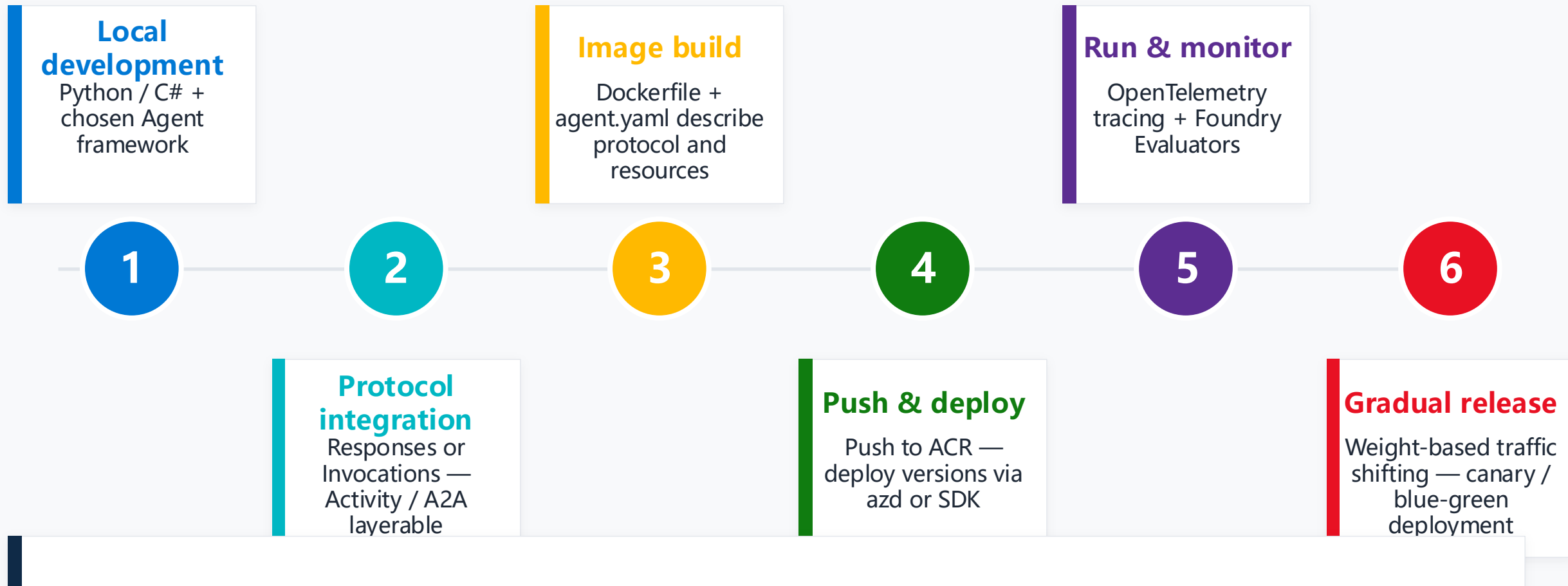
Practical guide

From dev workflow to context engineering, from performance optimization to security practices



Hosted Agent end-to-end dev workflow

Using Azure Foundry as example — a six-stage dev loop



6 principles for well-designed Agent Harness

Distilled engineering wisdom: balance stable interfaces and evolvable implementations

01 | Stable interfaces first

Design interfaces like OS abstractions — let the implementation evolve freely

02 | Decouple before coupling

Components fail, are replaced, and scale independently

03 | No pets

Treat containers, Brains, and Hands all as replaceable cattle

04 | Least privilege

Credentials never enter the sandbox — isolated via proxy or Vault

05 | Externalize state

Event log stored independently — recoverable, replayable, auditable

06 | Continuously question assumptions

After model upgrades, old Harness fallback logic may become dead code

Key techniques for context engineering

Bridge between Session and Context Window

Common techniques

Compaction

Compress context into summaries — periodically write back

Memory Tool

Claude proactively writes files — enabling cross-session learning

Context Trimming

Selectively delete old tool results or thinking blocks

Event Replay

Replay history slices via `getEvents()`

Cache optimization

Organize context to boost Prompt Cache hits

Golden rules

Session $\neq$ Context Window

- ▶ Session is recoverable event storage; Context is the current inference input.
- ▶ Leave irreversible decisions (compaction, trimming) to the Harness — keep Session intact.
- ▶ Use position-slice interfaces — let Brain return to any historical point.
- ▶ Harness can transform data freely before writing into Context.
- ▶ Future models may not need these techniques — interfaces must leave room for change.

Performance optimization: TTFT is the key user-felt metric

Time-To-First-Token decides how long the user feels they're waiting

What is TTFT?

Time-To-First-Token measures the latency from when a session accepts work to when it produces the first response Token. This is the latency users perceive most acutely.

Why does it slow down?

When Brain is placed inside the container, every session (including those that never invoke the sandbox) must first pay the cost of container startup, repo clone, and event pulling.

Anthropic's measured improvements after decoupling



Common pitfalls and how to avoid them

6 anti-patterns distilled from production practice



Pet-ifying containers

Scrambling to rescue on failure

Fix

Treat containers as restartable cattle



Credentials in the sandbox

Prompt Injection can steal Tokens

Fix

Store credentials in a Vault — call via proxy



Harness with embedded resource assumptions

Cannot connect to customer VPC

Fix

Fully decouple Harness from Hand



Irreversible compaction

Critical context can't be recovered after compaction

Fix

Persist event stream — defer decisions



Compatibility dead code

Workarounds for old models drag new models down

Fix

Periodically re-examine Harness assumptions



Debugging black box

WebSocket streams obscure the source of problems

Fix

Component-level tracing + isolated failure domains

Security best-practices checklist

Treat Agent Harness as production application code

Credential governance

- ✓ Never embed secrets in images
- ✓ Use Managed Identity
- ✓ Use Key Vault / Secret Manager for sensitive data

Network isolation

- ✓ Deploy Foundry resources in private network
- ✓ VNet / VPC peering only when needed
- ✓ Data crossing boundaries must be compliance-reviewed

Least privilege

- ✓ Each Agent has its own identity
- ✓ Fine-grained RBAC authorization
- ✓ OBO only when user delegation is required

Supply-chain security

- ✓ Image signing and vulnerability scanning
- ✓ MCP / third-party tool allowlists
- ✓ Dependency pinning and SBOM

Traceability

- ✓ Full Action audit log
- ✓ Tamper-proof event stream
- ✓ Anomaly alerting and replay

Prompt security

- ✓ Input sanitization + Output review
- ✓ Human confirmation for sensitive tools
- ✓ Prompt-Injection defense templates

Future trends: Meta-Harness and evolvable architecture

The next decade of Agent platforms: stable interfaces, free implementation

"Programs as yet unthought of."

Operating systems lasted decades by virtualizing hardware into sufficiently general abstractions. Managed Agents take the same road: opinionated about interfaces, neutral about implementations.

Standardization of universal Agent interfaces

Protocol layers like MCP / A2A / OpenAI Responses will keep converging

Agent as infrastructure

Agents become first-class citizens of cloud platforms — alongside VMs, containers, and functions

Distributed Brain-Hand deployment

Brain across clouds, Hand inside the customer — hybrid-cloud Agents

Evaluation and security closed loop

Evaluators + audit + policy unified — production-grade governance

T H A N K Y O U

Wrap-up and Q&A

- Agent Harness converges all engineering complexity beyond LLM inference into the platform layer
- Brain / Hands / Session decoupling is the core design philosophy of Managed Agents
- Azure / Anthropic / AWS each have their strengths — selection should fit the business scenario
- Stable interfaces matter more than clever implementations — leave room for the future